

Pandas LEVEL 5 — Indexing

Line-by-line examples, outputs, and easy interview explanations

1. What is Index?

Index is the label used to identify each row in a Pandas Series or DataFrame.

Think of an index like a row number or ID. It helps Pandas locate and select rows.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [21, 22, 23]
})

print(df)
```

Output:

	Name	Age
0	Amit	21
1	Riya	22
2	John	23

0, 1, 2 are the index labels.

2. Default Index

When you create a DataFrame without specifying an index, Pandas automatically creates a default integer index starting from 0.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
})

print(df.index)
print(df)
```

Output:

```
RangeIndex(start=0, stop=3, step=1)
```

	Name	Marks
0	Amit	80
1	Riya	90
2	John	85

3. Custom Index

You can give your own labels to rows instead of using 0, 1, 2.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame(
    {
        "Name": ["Amit", "Riya", "John"],
        "Marks": [80, 90, 85]
    },
    index=["S1", "S2", "S3"]
)

print(df)
```

Output:

	Name	Marks
S1	Amit	80
S2	Riya	90
S3	John	85

4. Changing Index

You can replace the existing index with a new list of labels. The number of new labels must match the number of rows.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
})

df.index = ["S1", "S2", "S3"]

print(df)
```

Output:

	Name	Marks
S1	Amit	80
S2	Riya	90
S3	John	85

5. set_index()

set_index() makes an existing column the DataFrame index. By default, that column is removed from the normal columns.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
})

df = df.set_index("ID")

print(df)
```

Output:

	Name	Marks
ID		
101	Amit	80
102	Riya	90
103	John	85

6. set_index() with inplace=True

`inplace=True` changes the original DataFrame directly. Without `inplace=True`, `set_index()` returns a new DataFrame.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Riya", "John"]
})

df.set_index("ID", inplace=True)

print(df)
```

Output:

	Name
ID	
101	Amit
102	Riya
103	John

7. reset_index()

`reset_index()` changes the current index back into a normal column and creates a new default index 0, 1, 2, ...

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Riya", "John"]
})

df = df.set_index("ID")
df = df.reset_index()

print(df)
```

Output:

	ID	Name
0	101	Amit
1	102	Riya
2	103	John

8. reset_index(drop=True)

drop=True removes the old index instead of converting it into a column. This is useful when you only want a fresh 0-based index.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"]
}, index=["A", "B", "C"])

df = df.reset_index(drop=True)

print(df)
```

Output:

```
   Name
0  Amit
1  Riya
2  John
```

9. Index Selection

You can select rows using their index labels with loc[].

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
}, index=["S1", "S2", "S3"])

print(df.loc["S2"])
```

Output:

```
Name      Riya
Marks      90
Name: S2, dtype: object
```

10. Select Multiple Index Labels

loc[] can select multiple rows by giving a list of index labels.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
}, index=["S1", "S2", "S3"])

print(df.loc[["S1", "S3"]])
```

Output:

```
   Name  Marks
S1  Amit     80
S3  John     85
```

11. Index Slicing

Index slicing selects a range of rows. With label-based slicing using `loc[]`, both the start and end labels are included.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Marks": [80, 90, 85, 95]
}, index=["S1", "S2", "S3", "S4"])

print(df.loc["S2":"S4"])
```

Output:

	Name	Marks
S2	Riya	90
S3	John	85
S4	Neha	95

12. Index Slicing with `iloc[]`

`iloc[]` selects rows by integer position. Here, positions start from 0. The stop position is excluded, just like normal Python slicing.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Marks": [80, 90, 85, 95]
}, index=["S1", "S2", "S3", "S4"])

print(df.iloc[1:3])
```

Output:

	Name	Marks
S2	Riya	90
S3	John	85

13. Duplicate Indexes

Pandas allows duplicate index labels. However, selecting a duplicated label can return multiple rows.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Marks": [80, 90, 85]
}, index=["S1", "S1", "S2"])

print(df)
print()
print(df.loc["S1"])
```

Output:

	Name	Marks
S1	Amit	80
S1	Riya	90
S2	John	85

	Name	Marks
S1	Amit	80
S1	Riya	90

14. Check for Duplicate Indexes

`index.duplicated()` tells you which index labels are duplicates. True means that the label has appeared before.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"]
}, index=["S1", "S1", "S2"])

print(df.index.duplicated())
```

Output:

```
[False  True False]
```

15. MultiIndex Introduction

A MultiIndex is an index with more than one level. It is useful when data has multiple grouping levels, such as Department and Student.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame({
    "Marks": [80, 90, 85, 95]
}, index=[
    ["IT", "IT", "HR", "HR"],
    ["Amit", "Riya", "John", "Neha"]
])

df.index.names = ["Department", "Student"]

print(df)
```

Output:

		Marks
Department	Student	
IT	Amit	80
	Riya	90
HR	John	85
	Neha	95

16. Creating MultiIndex with `from_tuples()`

`MultiIndex.from_tuples()` creates a MultiIndex from pairs or tuples.

Code — line by line:

```
import pandas as pd

index = pd.MultiIndex.from_tuples(
    [
        ("IT", "Amit"),
        ("IT", "Riya"),
        ("HR", "John")
    ],
    names=["Department", "Student"]
)

df = pd.DataFrame({"Marks": [80, 90, 85]}, index=index)

print(df)
```

Output:

		Marks
Department	Student	
IT	Amit	80
	Riya	90
HR	John	85

17. Selecting from MultiIndex

loc[] can select data using one level or multiple levels of a MultiIndex.

Code — line by line:

```
import pandas as pd

index = pd.MultiIndex.from_tuples(
    [("IT", "Amit"), ("IT", "Riya"), ("HR", "John")],
    names=["Department", "Student"]
)

df = pd.DataFrame({"Marks": [80, 90, 85]}, index=index)

print(df.loc["IT"])
```

Output:

	Marks
Student	
Amit	80
Riya	90

18. Selecting One Exact MultiIndex Row

Give both index labels as a tuple to select one exact row.

Code — line by line:

```
import pandas as pd

index = pd.MultiIndex.from_tuples(
    [("IT", "Amit"), ("IT", "Riya"), ("HR", "John")],
    names=["Department", "Student"]
)

df = pd.DataFrame({"Marks": [80, 90, 85]}, index=index)

print(df.loc[("IT", "Amit")])
```

Output:

```
Marks      80
Name: (IT, Amit), dtype: int64
```

19. Interview Difference: loc vs iloc

loc[] = label-based selection. iloc[] = position-based selection.

Example: If the index is ["S1", "S2", "S3"]: df.loc["S2"] selects the row labeled S2. df.iloc[1] selects the second row by position.

Code — line by line:

```
import pandas as pd

df = pd.DataFrame(
    {"Name": ["Amit", "Riya", "John"]},
    index=["S1", "S2", "S3"]
)

print(df.loc["S2"])
print()
print(df.iloc[1])
```

Output:

```
Name      Riya
Name: S2, dtype: object
```

```
Name      Riya
Name: S2, dtype: object
```

20. Quick Revision

Index = row label.

Default index = 0, 1, 2, ...

Custom index = user-defined labels.

set_index() = make a column the index.

reset_index() = convert index back to a column and create a default index.

loc[] = select by label.

iloc[] = select by position.

Duplicate indexes = same label can appear more than once.

MultiIndex = multiple levels of row indexes.

Code — line by line:

```
# Most important commands

df.index
df.index = ["A", "B", "C"]
df.set_index("ID")
df.reset_index()
df.reset_index(drop=True)
df.loc["A"]
df.loc[["A", "C"]]
df.loc["A":"C"]
```



```
df.iloc[0]  
df.iloc[0:2]  
df.index.duplicated()
```

Output:

These commands cover the core Pandas Level 5 indexing concepts.

Level 5 — Interview Questions

Q1. What is an index in Pandas?

An index is a label used to identify rows in a Series or DataFrame.

Q2. What is the default index?

Pandas normally creates 0, 1, 2, 3, ... when no custom index is provided.

Q3. What does `set_index()` do?

It makes an existing column the row index.

Q4. What does `reset_index()` do?

It converts the current index into a column and creates a default integer index.

Q5. What is the difference between `loc` and `iloc`?

`loc` uses labels; `iloc` uses integer positions.

Q6. Can Pandas have duplicate indexes?

Yes. Pandas allows duplicate index labels.

Q7. What is `MultiIndex`?

`MultiIndex` allows multiple levels of row labels, such as Department and Student.